

Portfolio Optimization with Black-Litterman

A Case Study in Public Portfolio Disclosures

Walk-forward backtest · Real SEC & congressional filing data · Buffett · Pelosi · Trump

Abstract. This report describes a Python codebase that applies a Black-Litterman portfolio overlay to publicly disclosed equity holdings and evaluates it in a rolling walk-forward backtest. Three case studies are constructed from real regulatory filings — Berkshire Hathaway SEC 13F (Warren Buffett), U.S. House Financial Disclosure (Nancy Pelosi), and OGE 278e (Donald Trump) — and three strategies are compared: a static disclosed-weight portfolio, a sample mean-variance optimiser, and a Black-Litterman posterior fed into a long-only quadratic programme. The framework is fully YAML-configurable, ships with 63 unit and integration tests, and produces four analysis notebooks covering data quality, strategy comparison, hyperparameter sensitivity, and benchmark attribution.

Contents

1

Motivation and Research Question

2

2

Data Pipeline

2

2.1

Disclosures (data/disclosures.py)

3

2.2

Prices (data/prices.py)

3

3

Three Strategies

3

4

The Black-Litterman Model

4

4.1

Equilibrium Prior

4

4.2

Views from Disclosures

4

4.3

Posterior Update

4

5

Walk-Forward Backtest Engine

5

5.1

Design and Lookahead-Bias Prevention

5

5.2

Configuration

5

5.3

Output

5

6

Performance Metrics

5

7

Analysis Notebooks

6

7.1

Notebook 01 — Data Quality & Universe Overview

6

7.2

Notebook 02 — Strategy Comparison Case Study

6

7.3

Notebook 03 — Black-Litterman Sensitivity Analysis

6

7.4	Notebook 04 — Benchmark Attribution & Alpha Decomposition	7
8	Codebase Architecture	7
8.1	Package Layout	7
8.2	Quality and Observability	8
8.3	CLI Quick-Start	8
9	Known Limitation and Extensions	9
10	Summary	10

1 Motivation and Research Question

Three distinct regulatory filing regimes require institutional investors and public officials to disclose their equity holdings: the SEC 13F form (institutional investment managers with $\geq \$100\text{M}$ AUM), the U.S. House Financial Disclosure report (members of Congress), and the U.S. Office of Government Ethics 278e annual disclosure (executive-branch officials). Although these filings differ in granularity and timing, each provides a public record of approximate portfolio weights that can serve as an *investor view* within a quantitative portfolio construction framework.

Person	Filing type	Source
Warren Buffett	SEC 13F	Berkshire Hathaway, as of 2025-12-31
Nancy Pelosi	House FD report	Report 10066169; midpoints of ranges
Donald Trump	OGE 278e	Annual disclosure; midpoints / lower bounds

Range-based values (which appear in both House FD and OGE 278e filings) are converted to midpoints where both bounds are available, and to the reported lower bound otherwise.

Can a Black-Litterman overlay improve portfolio quality relative to:

- (1) the **disclosed** portfolio itself (passive replication)?
- (2) a **mean-variance** (sample-estimated Markowitz) baseline?

Improvement is assessed on annualised Sharpe ratio, Sortino ratio, maximum drawdown, HHI concentration, and average turnover.

2 Data Pipeline

The pipeline ingests two CSV files — holdings disclosures and adjusted daily prices — and produces a `CaseStudyResult` dataclass containing the three `BacktestResult` objects, one per strategy. Figure 1 summarises the flow.

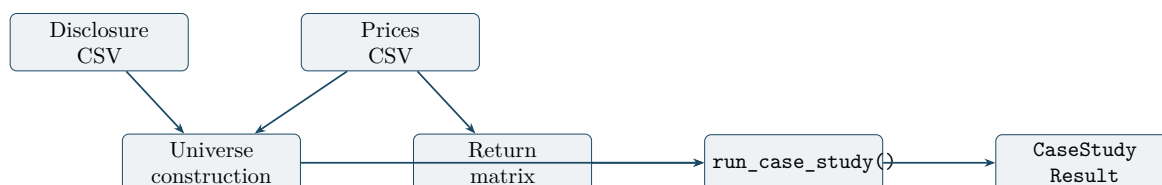


Figure 1: End-to-end data pipeline. Both CSV files are validated on load before any modelling occurs.

2.1 Disclosures (data/disclosures.py)

`load_disclosures_csv()` schema-validates the input on load, dropping any rows with missing or non-positive weights and emitting a warning for each dropped row.

`latest_portfolio_for_aliases()` selects the most recent filing date for a given set of ticker aliases, ensuring that the disclosed weights reflect the latest available snapshot without introducing any look-ahead bias relative to the price history.

2.2 Prices (data/prices.py)

The price dataset covers 46 tickers from 2018-01-02 to 2025-12-30 using split- and dividend-adjusted daily closing prices sourced from Yahoo Finance. `load_prices_csv()` detects and warns on duplicate `(date, ticker)` pairs. `to_return_matrix()` pivots the long-format price table into a wide daily log-return matrix aligned to the backtest calendar.

3 Three Strategies

All three strategies share the same asset universe, lookback window, rebalance schedule, and long-only constraint, ensuring that any observed performance differences are attributable to the portfolio construction method alone.

Strategy	Description
Disclosed	Static buy-and-hold of the filed portfolio weights. No rebalancing occurs; turnover is zero. This represents an upper bound on what a fully passive follower of the public disclosure can achieve.
Mean-Variance	Long-only Markowitz optimisation using the sample mean and covariance estimated over the lookback window. This is the classical quantitative benchmark, known to be sensitive to estimation error in the mean.
Black-Litterman	Equilibrium prior blended with disclosure-implied views to form a posterior expected return vector and covariance matrix, which are then passed to the same long-only quadratic programme. This is the primary strategy under investigation.

4 The Black-Litterman Model

4.1 Equilibrium Prior

The Black-Litterman framework begins by computing the *implied equilibrium excess return vector* by reverse-engineering the market-clearing condition. Given market-cap proxy weights $\mathbf{w}_{\text{mkt}}$, risk-aversion scalar λ , and asset covariance matrix Σ :

$$\boldsymbol{\pi} = \lambda \Sigma \mathbf{w}_{\text{mkt}} \quad (1)$$

with $\lambda = 2.5$ (default). These equilibrium returns serve as the prior mean for the distribution of expected returns.

4.2 Views from Disclosures

The disclosed portfolio weights define a single *absolute view*: the investor's filed holdings implicitly express a belief about relative expected returns. The view-picking matrix P ($k \times n$) extracts the relevant linear combination of assets, and the view return vector $\mathbf{q}$ encodes the expected return of that combination. View uncertainty is modelled as proportional to the prior covariance:

$$\Omega = \frac{1-c}{c} P \Sigma P^\top \quad (2)$$

where $c \in (0, 1)$ is the **view_confidence** parameter (default: 0.65). As $c \rightarrow 1$ the posterior collapses to the disclosed view; as $c \rightarrow 0$ it collapses to the equilibrium prior.

4.3 Posterior Update

Following He and Litterman (1999), the posterior is derived analytically. Define the posterior precision matrix:

$$M = (\tau \Sigma)^{-1} + P^\top \Omega^{-1} P \quad (3)$$

where $\tau = 0.05$ (default) controls the uncertainty in the prior. The posterior mean and covariance are:

$$\boldsymbol{\mu}_{\text{BL}} = M^{-1} \left[(\tau \Sigma)^{-1} \boldsymbol{\pi} + P^\top \Omega^{-1} \mathbf{q} \right] \quad (4)$$

$$\Sigma_{\text{BL}} = \Sigma + M^{-1} \quad (5)$$

The pair $(\boldsymbol{\mu}_{\text{BL}}, \Sigma_{\text{BL}})$ is then passed to a long-only quadratic programme with the full-investment constraint $\mathbf{1}^\top \mathbf{w} = 1$, $\mathbf{w} \geq 0$. If the solver returns all non-positive weights the strategy falls back to equal-weight allocation and logs a warning.

Symbol	Meaning
λ	Risk-aversion scalar (default: 2.5)
τ	Prior uncertainty scalar (default: 0.05)
c	View confidence (default: 0.65; swept in Notebook 03)
P	$k \times n$ view-picking matrix
$\mathbf{q}$	k -vector of view returns
Ω	$k \times k$ view uncertainty matrix

5 Walk-Forward Backtest Engine

5.1 Design and Lookahead-Bias Prevention

`rolling_backtest()` in `backtest/engine.py` implements a strict no-lookahead-bias design: at each month-end rebalance date t , portfolio weights are computed using only the 6-month trailing window of returns ending at t , and those weights are applied to returns *starting from the period after t* . The `weight_history` output is indexed by decision date, making it straightforward to audit the lag.

5.2 Configuration

All hyperparameters are declared in `configs/case_studies.yaml` and loaded into a frozen `BacktestConfig` dataclass at runtime. No magic numbers appear in the source code.

Parameter	Value
<code>lookback_periods</code>	6 months
<code>rebalance_frequency</code>	Month-end (ME)
<code>risk_aversion</code>	$\lambda = 2.5$
<code>tau</code>	$\tau = 0.05$
<code>view_confidence</code>	$c = 0.65$

5.3 Output

`rolling_backtest()` returns a `BacktestResult` dataclass with three fields: `.returns` (daily return series), `.nav` (cumulative NAV indexed from 1.0), and `.weight_history` (DataFrame of rebalance-date weights).

6 Performance Metrics

All metrics are computed by `backtest/metrics.py`. The helper `infer_periods_per_year()` automatically detects daily, monthly, or quarterly frequency from the return index and emits a `UserWarning` when fewer than three observations are available.

Metric	Notes
Annual return	Geometric compounding from NAV series.
Annual volatility	Annualised standard deviation of daily returns.
Sharpe ratio	Excess return over annualised volatility; risk-free rate defaults to zero.
Sortino ratio	Excess return over downside deviation; returns <code>nan</code> (not $+\infty$) when no negative daily return exists in the window, avoiding misleading statistics on strongly trending periods.
Max drawdown	Maximum peak-to-trough decline in cumulative NAV.
HHI concentration	$\sum_i w_i^2$; ranges from $1/n$ (equal-weight) to 1.0 (fully concentrated).
Average turnover	Mean L_1 weight change per rebalance: $\frac{1}{T} \sum_t \ \mathbf{w}_t - \mathbf{w}_{t-1}\ _1$.

`summarize_strategy()` aggregates all seven metrics into a dict, which the notebooks collect into a combined summary DataFrame for side-by-side comparison across strategies and persons.

7 Analysis Notebooks

The repository ships four Jupyter notebooks. All notebooks iterate over the **Buffett**, **Pelosi**, and **Trump** case studies from a shared `results` dict — no person keys are hardcoded.

7.1 Notebook 01 — Data Quality & Universe Overview

Performs schema checks on the raw holdings and price CSVs, reports holdings coverage (fraction of disclosed tickers with matching price history), and renders portfolio composition pie charts for each person. The output provides a sanity check that the data pipeline has ingested the filings correctly before any modelling is run.

7.2 Notebook 02 — Strategy Comparison Case Study

The main results notebook. For each person it plots:

- Equity curves (cumulative NAV) for all three strategies on a shared axis.
- Drawdown time series highlighting the 2020 COVID drawdown and 2022 rate-shock episodes.
- Black-Litterman weight evolution over rebalance dates, showing how the posterior allocation shifts as the disclosure-implied view interacts with the changing equilibrium prior.
- A combined summary table with all seven metrics for all three strategies.

7.3 Notebook 03 — Black-Litterman Sensitivity Analysis

Sweeps the view confidence parameter $c \in \{0.20, 0.25, \dots, 0.95\}$ (16 values) across all three persons, running 48 independent backtests in total. The results are displayed in a 2×3 subplot

grid showing how annualised return, volatility, Sharpe ratio, HHI, and average turnover respond to c .

Key findings from the sweep:

- The Sharpe ratio is *non-monotonic* in c on real data — the value of c that maximises Sharpe differs across persons and cannot be determined a priori.
- Higher confidence causes turnover to increase when the disclosed view diverges substantially from the equilibrium prior.
- HHI spikes near extreme values of c , indicating that very high or very low confidence produces concentrated allocations.

The best-by-Sharpe row is printed for each person, providing a principled starting range for `view_confidence` before any out-of-sample reporting.

7.4 Notebook 04 — Benchmark Attribution & Alpha Decomposition

Fits an OLS factor model to each strategy's return series:

$$r_{\text{strat},t} = \alpha + \beta_1 r_{\text{SPY},t} + \beta_2 r_{\text{Tech},t} + \beta_3 r_{\text{Energy},t} + \beta_4 r_{\text{Fin},t} + \beta_5 r_{\text{CD},t} + \varepsilon_t \quad (6)$$

where benchmarks are sourced preferentially from ETF tickers (SPY, XLK, XLE, XLF, XLY) with an automatic fallback to ticker-proxy averages (e.g. AAPL + MSFT + NVDA for Tech) if the ETF is absent from the price file. A provenance table is printed per person.

For each person the notebook produces:

- A $\hat{\beta}$ bar chart (positive loadings in teal, negative in red).
- An arithmetic return contribution decomposition: $\bar{r}_{\text{ann}} \approx \hat{\alpha} \cdot f + \sum_k \hat{\beta}_k \bar{r}_{k,\text{ann}} + \text{residual}$, where f is periods per year.
- A return correlation heatmap between the strategy and each factor.
- Rolling 12-month $\hat{\alpha}$ and $\hat{\beta}$ versus SPY, revealing regime shifts hidden in the full-sample statistics — most visibly during the 2020 COVID drawdown and the 2022 rate shock.

8 Codebase Architecture

8.1 Package Layout

The `portfolio_bl` package lives entirely under `src/` and is organised into three sub-packages.

Module	Responsibility
<code>config.py</code>	YAML loading; <code>BacktestConfig</code> and <code>CaseStudyConfig</code> frozen dataclasses.
<code>pipeline.py</code>	<code>run_case_study()</code> orchestrator; returns <code>CaseStudyResult</code> .
<code>data/disclosures.py</code>	Load, validate, and filter holdings CSV.
<code>data/prices.py</code>	Load prices CSV; pivot to return matrix.
<code>models/black_litterman.py</code>	<code>implied_equilibrium_returns()</code> ; <code>black_litterman_posterior()</code> .
<code>models/mean_variance.py</code>	<code>long_only_markowitz()</code> with equal-weight fallback.
<code>backtest/engine.py</code>	<code>rolling_backtest()</code> walk-forward loop.
<code>backtest/metrics.py</code>	All seven performance metrics; <code>summarize_strategy()</code> .

8.2 Quality and Observability

- **63 tests** (unit and integration) runnable with `pytest -tb=short`. Tests cover the BL posterior algebra, backtest engine edge cases (empty windows, all-positive returns), and metric correctness.
- `logging.getLogger(__name__)` is configured in every module. The `-verbose` flag elevates the log level to `DEBUG`, printing per-rebalance diagnostics without modifying any source file.
- `FileNotFoundError` and `ValueError` are raised with descriptive messages on bad or missing inputs; no silent failures.
- `__all__` is defined in every `__init__.py`, making the public API explicit.
- All BL hyperparameters are exposed via YAML; no magic numbers appear in the source code.

8.3 CLI Quick-Start

```
PYTHONPATH=src python scripts/run_case_study.py \
  --config configs/case_studies.yaml \
  --person buffett \
  --verbose
```

Swapping `-person buffett` for `pelosi` or `trump` runs the identical pipeline on a different case study. All three persons can also be run programmatically by calling `run_case_study()` in a loop with different `CaseStudyConfig` objects.

9 Known Limitation and Extensions

The BL posterior assumes multivariate Gaussianity in both returns and views. Equity return distributions exhibit significant excess kurtosis and left-tail asymmetry, particularly during stress periods directly visible in this backtest — the **2020 COVID drawdown** and the **2022 rate shock**. A production-grade system would replace the Gaussian prior with Meucci's Entropy Pooling / Copula Opinion Pooling framework, which separates the marginal distributions from the dependence structure and allows views to be expressed on arbitrary statistics including implied volatility.

The practical consequences within this backtest are:

- The posterior covariance Σ_{BL} underestimates tail risk during crisis periods, causing the optimiser to over-allocate to assets that appear low-volatility under normality but exhibit fat tails under stress.
- The Sortino ratio and maximum drawdown are therefore the most informative diagnostics in this framework, as they directly capture downside behaviour without assuming symmetry.

The Entropy Pooling approach is the natural upgrade path: it preserves the Bayesian blending intuition of Black-Litterman while removing the Gaussian assumption entirely.

10 Summary

What the framework provides

- End-to-end walk-forward backtest on real, publicly-filed portfolio data — not synthetic or simulated holdings.
- Fair three-way strategy comparison: Disclosed, Mean-Variance, and Black-Litterman on an identical universe and schedule.
- Full sensitivity sweep over the key BL hyperparameter (c), producing 48 backtests across three persons.
- OLS factor attribution with rolling 12-month diagnostics.
- YAML-configurable — swap in any universe or parameter set without modifying source code.
- 63 tests enforcing correctness at every layer of the stack.

Key design decisions

- No-lookahead-bias by construction in the engine.
- Graceful degradation to equal-weight with a logged warning rather than a hard crash.
- `nan` (not $+\infty$) Sortino on all-positive return periods.
- Real adjusted prices (2018–2025, 46 tickers) via Yahoo Finance.